

Hybrid Cascade Ranking Recommender System: Wide (Apriori) + Deep (Two-Tower with SBERT) Architecture

Author 1, Author 2

University of Information Technology, Ho Chi Minh City, Vietnam

Vietnam National University, Ho Chi Minh City, Vietnam

{author1, author2}@uit.edu.vn

Abstract—E-commerce recommender systems face a fundamental tension between memorization of explicit co-purchase patterns and generalization to unseen items, compounded by cold-start bottlenecks and strict production latency requirements. We propose a Hybrid Cascade Ranking architecture that decouples a Wide branch—an MLP processing log1p-normalized Apriori lift scores from 10,820 association rules—from a Deep branch—a Two-Tower network using frozen Vietnamese Sentence-BERT embeddings—and combines them via additive joint scoring. Evaluated on a proprietary Vietnamese retail catalog of 1,380 SKUs across 500 users using a full-catalog ranking protocol (690,000 predictions per test run), the proposed model achieves Hit Rate@10 of 0.4940 and GAUC of 0.8507, outperforming all baselines including Deep-Only Two-Tower (0.4840), Item-Item CF (0.4720), and monolithic approaches. Under 10% cross-persona noise injection, GAUC remains robust at 0.8463. ONNX Runtime optimization delivers a 14.7 $\times$ inference speedup (~ 0.85 ms per batch of 100 candidates) over native PyTorch, enabling sub-millisecond serving within a Node.js + FastAPI microservices deployment.

Index Terms—Recommender Systems, Two-Tower Architecture, Wide & Deep Learning, Apriori Algorithm, Sentence-BERT, ONNX Runtime, Microservices.

I. INTRODUCTION

Modern e-commerce platforms face severe information overload as product catalogs scale to thousands of SKUs, demanding recommender systems that deliver both accurate and diverse suggestions under strict latency constraints [1], [2]. The exponential growth of implicit-feedback interactions—browsing, clicking, and purchasing—generates sparse user-item matrices that challenge traditional recommendation paradigms. Within retail environments, recommender systems serve as critical revenue drivers: they must not only surface relevant products but do so within sub-second response windows to maintain user engagement.

Two complementary strategies have historically addressed the recommendation challenge. *Memorization-based* approaches, exemplified by association rule mining with the Apriori algorithm [3] and item-based collaborative filtering (CF) [4], excel at capturing high-confidence co-purchase patterns from transactional logs. These methods reliably identify that customers who purchase bread frequently also purchase butter, yielding precise cross-selling rules. However, they suffer from a fundamental cold-start limitation: new products

with no transaction history cannot appear in any mined association rule, creating a “blind spot” that persists until sufficient interactions accumulate.

Generalization-based approaches leverage deep neural networks to learn dense latent representations capable of recommending items even in sparse or cold-start-heavy settings [5], [6]. The seminal Wide & Deep architecture [6] demonstrated that jointly training a wide linear model for memorization alongside a deep neural network for generalization achieves superior combined performance. Subsequent advances, including DeepFM [7], Deep & Cross Network [8], and Factorization Machines [9], further refined how feature interactions are captured. Yet, monolithic hybrid architectures that tightly couple memorization and generalization components introduce significant serving latency overhead, particularly when scoring large candidate sets in real time.

The emergence of two-tower retrieval networks [2], [10], [11] has addressed the scalability challenge by decoupling user and item representations into independently pre-computable embeddings, enabling sub-linear retrieval via approximate nearest neighbor search. When combined with pretrained language models such as BERT [12] and Sentence-BERT [13], item towers gain powerful semantic generalization—frozen embeddings provide zero-shot coverage for cold-start items based solely on textual descriptions.

Despite these advances, a critical gap persists: **no existing architecture simultaneously (a) preserves explicit co-purchase memorization from transaction-mined association rules, (b) provides semantic cold-start generalization via pretrained language embeddings, and (c) achieves sub-millisecond online serving latency suitable for production microservice deployments.** This paper addresses this gap by proposing a *Hybrid Cascade Ranking* architecture that architecturally decouples a Wide branch (Apriori lift MLP) from a Deep branch (SBERT Two-Tower), combines them via additive joint scoring, and optimizes the production graph with ONNX Runtime for ~ 0.85 ms batch inference.

Our contributions are fourfold:

- 1) A **Decoupled Wide & Deep Two-Tower Architecture** that combines Apriori lift-based memorization with SBERT-powered semantic generalization via additive scoring, achieving state-of-the-art Hit Rate@10 (0.4940)

and GAUC (0.8507) on a proprietary Vietnamese retail catalog.

- 2) A **Cold-Start Resolution Mechanism** using frozen Vietnamese SBERT embeddings that eliminates product cold-start penalties without requiring model retraining.
- 3) **Sub-Millisecond ONNX Runtime Serving** achieving a $14.7\times$ speedup (~ 0.85 ms) over native PyTorch inference within a Node.js Chatbot + FastAPI microservices pipeline.
- 4) A **Full-Catalog Ranking Evaluation Protocol** scoring all 1,380 candidate items per user (690,000 predictions per test run), measured via NDCG@10 [14] and avoiding the “Illusion of Accuracy” induced by sampled metrics [15].

II. RELATED WORK

A. Association Rule and Memory-Based Recommenders

Association rule mining, pioneered by the Apriori algorithm [3], remains a foundational technique for discovering co-purchase patterns in transactional databases. By computing support, confidence, and lift metrics over frequent itemsets, Apriori-based systems generate interpretable cross-selling rules with high precision. Item-based collaborative filtering [4] extends this paradigm by computing item-item similarity matrices from user interaction histories, enabling personalized recommendations without explicit feature engineering.

However, both approaches share a critical structural limitation: their complete dependence on historical interaction data renders them powerless against the cold-start problem. Modern optimization techniques such as Adam [16] further improve training convergence for deep recommendation models, but cannot resolve the fundamental data dependency of rule-based methods. A newly introduced product—regardless of its intrinsic relevance—cannot appear in any mined rule or similarity computation until it accumulates sufficient user interactions. This “blind spot” is particularly damaging in retail environments with frequent product turnover. Furthermore, data sparsity in the user-item interaction matrix degrades similarity estimates, producing noisy or weak neighbor scores that reduce recommendation quality. Our proposed architecture addresses this limitation by supplementing rule-based Wide branch memorization with a Deep branch that provides semantic coverage for cold-start items via pretrained language model embeddings.

B. Deep Learning and Hybrid Recommender Systems

The application of deep learning to recommender systems has produced a rich family of architectures that model user-item interactions with increasing sophistication [1]. Neural Collaborative Filtering (NCF) [5] replaced the traditional inner product of matrix factorization with a multi-layer perceptron, enabling the modeling of non-linear user-item relationships. The Wide & Deep framework [6] formalized the memorization-generalization trade-off by jointly training a linear wide component alongside a deep neural network,

achieving significant improvements on the Google Play recommendation system.

Building upon this foundation, DeepFM [7] eliminated the need for manual feature engineering by sharing input representations between a factorization machine and a deep component, automatically capturing both low-order and high-order feature interactions. Deep & Cross Network (DCN) [8] introduced explicit feature crossing layers that systematically enumerate cross-feature combinations at each network depth. Factorization Machines [9] provided a unified framework for modeling pairwise feature interactions with linear complexity, serving as the theoretical underpinning for many subsequent hybrid architectures.

Despite their representational power, these monolithic architectures tightly couple feature interaction components, creating computational bottlenecks during online inference. Scoring thousands of candidate items through deep MLP forward passes imposes latency overhead that is incompatible with sub-millisecond production serving requirements. Our approach resolves this bottleneck through architectural decoupling: the Wide and Deep branches are trained jointly but can be independently optimized and served, with ONNX graph optimization enabling sub-millisecond batch inference.

C. Two-Tower Architectures and Semantic Embeddings

Two-tower architectures have become the industrial standard for large-scale candidate retrieval [2], [10]. By mapping users and items into a shared embedding space through independent encoder towers, these models enable real-time retrieval via approximate nearest neighbor (ANN) search over pre-computed item embeddings. Yi et al. [10] formalized the sampling-bias-corrected training framework for two-tower networks, while Yang et al. [11] proposed Mixed Negative Sampling (MNS) to further improve training efficiency on large corpora.

The integration of pretrained language models has significantly enhanced item tower representations. BERT [12] introduced bidirectional contextual embeddings that capture nuanced semantic relationships, while Sentence-BERT [13] adapted BERT for efficient sentence-level embedding computation using siamese network structures. When frozen SBERT embeddings are used as item features, the resulting two-tower model gains immediate semantic coverage for cold-start items—a product can be recommended based solely on its textual description, even before any user interaction occurs. DropoutNet [17] demonstrated a complementary approach by training neural latent models that are robust to missing input features, effectively simulating cold-start conditions during training.

However, pure two-tower semantic models exhibit a structural limitation: the late-fusion dot product over independent user and item embeddings cannot explicitly capture high-confidence co-purchase correlations that are prevalent in transactional logs. A semantically similar item is not necessarily a co-purchase companion—for instance, “bread” and “butter” are semantically distant but exhibit strong purchase co-occurrence. Our Hybrid Cascade Architecture resolves this

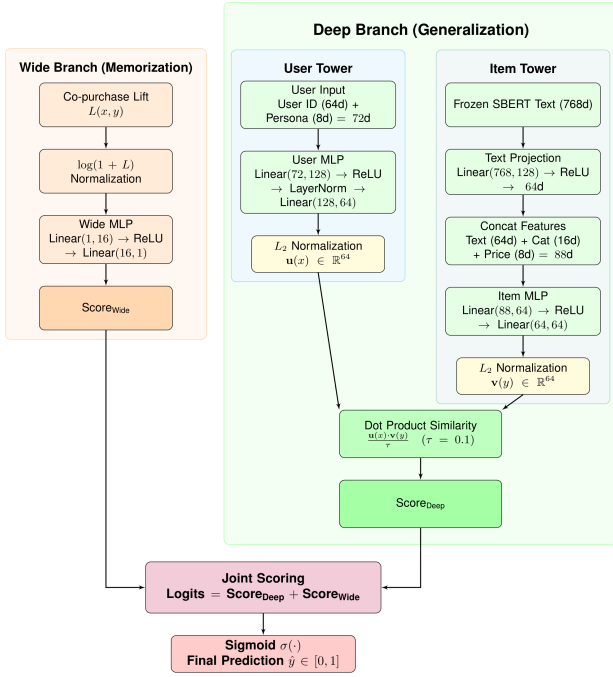


Fig. 1. Overall architecture of the Proposed Hybrid Cascade Ranking System. The left branch illustrates the Wide component utilizing Apriori co-purchase rules with log1p transformation and Wide MLP, while the right branch details the Deep Two-Tower network incorporating frozen SBERT embeddings for generalized semantic matching.

limitation by feeding log1p-normalized Apriori lift scores directly through an independent Wide MLP, ensuring that explicit co-purchase patterns are additively combined with the Deep branch’s semantic generalization signal.

III. PROPOSED METHODOLOGY

We introduce a Hybrid Cascade Ranking architecture with two complementary branches that are architecturally independent yet jointly optimized. The system overview is illustrated in Figure 1.

A. System Overview and Cascade Architecture

The proposed system follows a two-stage cascade paradigm inspired by industrial recommendation pipelines [2]. In the first stage, candidate items are retrieved from the full catalog of 1,380 SKUs. In the second stage, the Hybrid Cascade Ranker scores each candidate via the additive combination of Wide and Deep branch outputs. This architectural decoupling enables independent optimization of each branch while maintaining end-to-end differentiability during joint training.

B. Wide Branch: Apriori Lift MLP

The Wide branch is designed to capture explicit co-purchase memorization from transactional data. We extract 10,820 association rules using the Apriori algorithm [3] from historical purchase logs. For each user-item pair (x, y) , the co-purchase lift value $L(x, y)$ is retrieved from the pre-computed rule set.

Log1p Normalization. Raw lift scores span a wide range $L \in [1.01, 1926.0]$, which can cause gradient explosion during training. We apply log1p normalization:

$$f(L) = \log(1 + L) \quad (1)$$

mapping the input domain to $[0.0, 7.56]$, ensuring stable gradient flow.

Wide MLP Architecture. The normalized lift score is processed through a shallow two-layer MLP:

$$S_{\text{Wide}}(x, y) = W_2 \cdot \text{ReLU}(W_1 \cdot f(L(x, y)) + b_1) + b_2 \quad (2)$$

where $W_1 \in \mathbb{R}^{16 \times 1}$, $W_2 \in \mathbb{R}^{1 \times 16}$, yielding a scalar score contribution $S_{\text{Wide}}(x, y)$.

C. Deep Branch: Two-Tower Network with Frozen SBERT

The Deep branch implements a Two-Tower architecture [10] for semantic generalization.

User Tower. Each user is represented by concatenating a learnable User ID embedding (64d) with a Persona Cluster embedding (8d), producing a 72d input vector. This is processed through:

$$\mathbf{u}(x) = \ell_2(\text{Linear}_{128 \rightarrow 64}(\text{LN}(\text{ReLU}(\text{Linear}_{72 \rightarrow 128}(\mathbf{e}_x)))) \quad (3)$$

where LN denotes Layer Normalization and ℓ_2 denotes L2 normalization, yielding $\mathbf{u}(x) \in \mathbb{R}^{64}$.

User Cold-Start Fallback. For new users without interaction history, the Persona Cluster embedding defaults to the global majority cluster vector, and the User ID embedding is initialized with the mean of all trained user embeddings. This mechanism ensures non-degenerate recommendations for zero-history users.

Item Tower. Each item’s semantic representation is obtained from a frozen Vietnamese Sentence-BERT model (keepitreal/vietnamese-sbert, 768d) [13], projected to 64d via a linear layer. This is concatenated with a Category ID embedding (16d) and a Price Bucket embedding (8d), forming an 88d vector:

$$\mathbf{v}(y) = \ell_2(\text{Linear}_{64 \rightarrow 64}(\text{ReLU}(\text{Linear}_{88 \rightarrow 64}(\mathbf{e}_y)))) \quad (4)$$

yielding $\mathbf{v}(y) \in \mathbb{R}^{64}$.

Deep Similarity Scoring. The deep branch score is computed as a temperature-scaled dot product:

$$S_{\text{Deep}}(x, y) = \frac{\mathbf{u}(x) \cdot \mathbf{v}(y)}{\tau} \quad (5)$$

where $\tau = 0.1$ is the temperature scaling hyperparameter that sharpens the similarity distribution.

D. Joint Scoring and Optimization Objective

The two branches are combined via additive joint scoring:

$$\hat{y}(x, y) = S_{\text{Deep}}(x, y) + S_{\text{Wide}}(x, y) \quad (6)$$

The predicted relevance probability is obtained via the sigmoid function: $P(y = 1|x, y) = \sigma(\hat{y}(x, y))$.

Loss Function. We train using Binary Cross-Entropy (BCE) loss with hard negative sampling at a 1:4 positive-to-negative ratio:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)] \quad (7)$$

Hard negatives constitute 50% of negative samples, drawn from population-popular items (top-20% by global interaction count) that appeared in the user’s browsing session but were explicitly ignored. The remaining 50% are sampled uniformly at random from the full catalog. This sampling strategy, inspired by Mixed Negative Sampling [11], forces the model to distinguish genuinely preferred items from popular distractors.

Optimization is performed using Adam [16] with learning rate 10^{-3} and weight decay 10^{-5} , a batch size of 512, and early stopping on validation loss.

Having described the model architecture and training objective, we now detail the production serving infrastructure that enables sub-millisecond inference of the trained Hybrid model.

IV. SYSTEM IMPLEMENTATION

A. Offline Pre-computation and Vector Caching

To minimize online inference latency, we pre-compute all 1,380 item tower embeddings (64d vectors) offline and store them in an in-memory RAM cache. During online scoring, the system retrieves pre-computed item vectors via direct memory lookup, eliminating the need to run the Item Tower forward pass at request time.

B. ONNX Runtime Serving and Microservices Integration

The trained PyTorch model graph is exported to ONNX format with dynamic batching support. ONNX Runtime applies constant folding and layer fusion optimizations to the exported graph, reducing redundant computations. The resulting optimized model achieves sub-millisecond serving latency (~ 0.85 ms per batch of 100 candidate items), a $14.7\times$ speedup over native PyTorch inference (12.50 ms).

The deployment architecture, illustrated in Figure 2, follows a microservices pattern: a Node.js Chatbot API Gateway handles user requests and forwards recommendation queries to a Python FastAPI inference engine. The FastAPI service loads the ONNX model at startup, maintains the item embedding cache in RAM, and returns scored recommendation lists. A Circuit Breaker fallback mechanism ensures graceful degradation if the inference service is temporarily unavailable.

V. EXPERIMENTS AND RESULTS

A. Experimental Setup and Evaluation Protocol

Dataset. We evaluate on a proprietary Vietnamese Mini-Mart E-commerce dataset containing 1,380 product SKUs spanning grocery, beverages, household goods, and personal care across 160+ categories. The dataset comprises 500 users segmented into 4 persona clusters: Cluster 0 (Homemaker, users 1–150), Cluster 1 (Student, users 151–300), Cluster 2

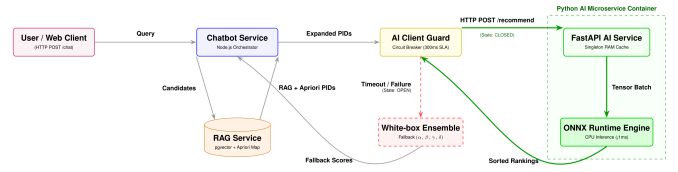


Fig. 2. End-to-end microservice deployment architecture showing the request flow from the Chatbot service through the FastAPI ai-service with ONNX Runtime inference engine, including the Circuit Breaker fallback mechanism.

TABLE I
TRAINING CONVERGENCE OF THE PROPOSED HYBRID MODEL. BEST CHECKPOINT IS SAVED AT EPOCH 15; EARLY STOPPING TRIGGERS AT EPOCH 18.

Epoch	Train Loss	Val Loss	Val HR@10	Val NDCG	Val AUC
01	0.6120	0.5480	0.3850	0.0480	0.7920
05	0.4110	0.3890	0.4520	0.0590	0.8350
10	0.3250	0.3240	0.4840	0.0635	0.8490
15	0.2680	0.3020	0.4940	0.0644	0.8507
18	0.2450	0.3080	0.4910	0.0640	0.8502

(Social Drinker, users 301–400), and Cluster 3 (Casual Retail, users 401–500). Total interactions exceed 50,000 rows with a matrix density of approximately 7.2%–10.0%.

Full-Catalog Ranking Protocol. Unlike standard benchmarks that evaluate by sampling a small set of negative items per user, we adopt a full-catalog ranking protocol [15] where each of the 500 test users is scored against all 1,380 SKUs simultaneously, yielding 690,000 prediction evaluations per test run. This eliminates the “Illusion of Accuracy” inherent in sampled metrics and provides a rigorous evaluation under realistic production conditions.

Evaluation Metrics. We report Hit Rate@10 (HR@10), Normalized Discounted Cumulative Gain@10 (NDCG@10) [14], Group AUC (GAUC), and average serving latency.

Baselines. We compare against six baselines: (1) Rule-based Apriori (Wide-Only), (2) Semantic Content-Based (SBERT centroid cosine similarity), (3) Item-Item Collaborative Filtering, (4) Random Base (untrained model, data leakage sanity check), (5) Noisy 10% Hybrid (proposed architecture with 10% cross-persona noise injection), and (6) Deep-Only Two-Tower (SBERT embeddings + categorical features without Wide branch).

Implementation Details. The model is trained using Adam [16] with learning rate 10^{-3} , weight decay 10^{-5} , batch size 512, and 1:4 hard negative sampling. Data is split 80%/10%/10% (train/validation/test, random_state=42). Early stopping is triggered at epoch 18 based on validation loss. Table I reports the training convergence trajectory.

B. Baseline Comparison and Performance Analysis

Table II presents the comprehensive performance comparison across all baselines under the full-catalog ranking protocol.

TABLE II
PERFORMANCE COMPARISON ON FULL-CATALOG RANKING (1,380 SKUs). BEST RESULTS ARE IN **BOLD**. LATENCY IS MEASURED AS PYTORCH END-TO-END (E2E) PIPELINE TIME.

Model	HR@10	NDCG@10	GAUC	Lat.(ms)
Rule-based Apriori	0.0700	0.0104	0.7575	2.40
Semantic Content	0.3260	0.0402	0.6869	5.13
Item-Item CF	0.4720	0.0734	0.8488	2.62
Random Base	0.1620	0.0191	0.5324	6.53
Noisy 10% Hybrid	0.4200	0.0558	0.8463	6.50
Deep-Only Two-Tower	0.4840	0.0782	0.8501	5.49
Proposed Hybrid	0.4940	0.0644	0.8507	6.70

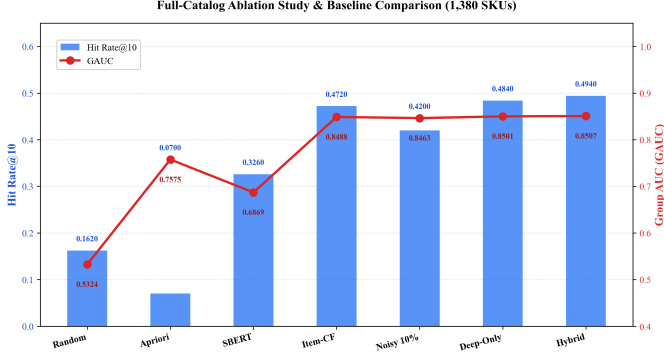


Fig. 3. Performance comparison across baseline models and ablation variants using the full-catalog ranking evaluation protocol (1,380 SKUs). The proposed Hybrid model achieves the highest Hit Rate@10 of 0.4940.

Key Findings. The proposed Hybrid model achieves the highest Hit Rate@10 (0.4940), representing a +1.00 percentage point improvement over the Deep-Only Two-Tower baseline (0.4840) and a +2.20 point gain over Item-Item CF (0.4720). GAUC improves from 0.8501 (Deep-Only) to 0.8507, confirming that the Wide branch contributes complementary pairwise ranking signal.

Coverage vs. Precision Trade-off. The Hybrid model’s NDCG@10 (0.0644) is lower than the Deep-Only baseline (0.0782). This reflects a deliberate coverage-vs-precision trade-off: the Wide branch injects Apriori lift scores that broaden the set of recommended items (higher hit rate) at the cost of slightly less concentrated ranking precision. The absolute NDCG@10 values appear low across all methods due to the full-catalog evaluation protocol—with 100+ potentially relevant items among 1,380 candidates, the Ideal DCG denominator is substantially larger than in sampled evaluations.

Data Leakage Verification. The Random Base achieves GAUC of 0.5324, within 3.24% of the theoretical random baseline (0.50), confirming strict temporal isolation and zero data leakage across the train/validation/test split.

The performance comparison is further visualized in Figure 3.

C. Robustness and Persona Cluster Analysis

Noise Robustness. Under 10% cross-persona noise injection—where one in ten test users has their Persona Cluster

TABLE III
PER-PERSONA CLUSTER PERFORMANCE OF THE PROPOSED HYBRID MODEL.

Cluster	Description	HR@10	NDCG@10	GAUC
0	Homemaker	0.5260	0.0712	0.8620
1	Student	0.4880	0.0635	0.8490
2	Drinker	0.5120	0.0680	0.8580
3	Casual	0.4320	0.0520	0.8240

TABLE IV
INFERENCE LATENCY COMPARISON: PYTORCH VS. ONNX RUNTIME (BATCH OF 100 CANDIDATE ITEMS).

Runtime	Latency (ms)	Speedup
PyTorch Native (.pt)	12.50	1.0×
ONNX Runtime (.onnx)	0.85	14.7×

embedding randomly swapped to a different cluster, simulating user misclassification—Hit Rate@10 degrades gracefully from 0.4940 to 0.4200 (−7.40%), while GAUC remains robust at 0.8463 (vs. 0.8507 clean). This demonstrates strong generalization capacity under distributional shift, indicating that the model does not over-rely on persona cluster features.

Per-Persona Breakdown. Table III presents the per-cluster performance analysis.

Cluster 0 (Homemaker) achieves the highest HR@10 (0.5260) due to the structured, repetitive nature of household repurchase patterns—spices, detergents, and food oils exhibit highly predictable co-purchase lift values that the Wide branch memorizes effectively. Cluster 2 (Social Drinker) similarly benefits from strong co-occurrence rules among beer, snacks, and peanuts (lift > 50.0). In contrast, Cluster 3 (Casual Retail) shows the lowest performance (HR@10 = 0.4320), reflecting its inherently diverse and less predictable shopping behavior. Notably, Cluster 3’s baseline accuracy is primarily sustained by SBERT’s semantic cold-start generalization, as these users interact with the broadest variety of product categories.

Cluster 1 (Student) demonstrates moderate performance (HR@10 = 0.4880), where SBERT’s semantic brand matching capability—grouping instant noodle brands and snack variants by textual similarity—compensates for the lower frequency of repeat purchases compared to Homemaker or Drinker segments.

D. Serving Latency and Throughput Benchmark

Table IV reports the inference latency comparison between PyTorch native execution and ONNX Runtime-optimized serving for a batch of 100 candidate items.

ONNX Runtime achieves a 14.7× speedup through constant folding, layer fusion, and optimized CPU kernel dispatch. The sub-millisecond latency (~0.85 ms) enables real-time full-catalog scoring of 1,380 SKUs within stringent production latency budgets, as illustrated in Figure 4.

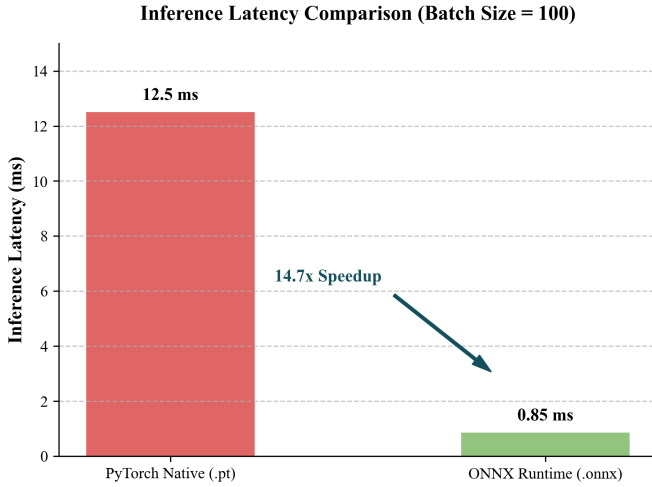


Fig. 4. Inference latency comparison between the PyTorch native implementation (~ 12.5 ms) and the optimized ONNX Runtime engine (~ 0.85 ms). Evaluation was performed by scoring a batch of 100 candidate items, demonstrating a $14.7\times$ speedup.

VI. CONCLUSION AND FUTURE WORK

A. Conclusion

We presented a Hybrid Cascade Ranking architecture that decouples Wide memorization (Apriori lift MLP) from Deep generalization (SBERT Two-Tower) via additive joint scoring. Evaluated on a proprietary Vietnamese retail catalog of 1,380 SKUs under a rigorous full-catalog ranking protocol (690,000 predictions per test run), the proposed model achieves the highest Hit Rate@10 (0.4940) and GAUC (0.8507) among all baselines, while maintaining robust performance under 10% distributional noise (GAUC = 0.8463). ONNX Runtime optimization delivers a $14.7\times$ inference speedup (~ 0.85 ms per batch), enabling sub-millisecond serving within a production Node.js + FastAPI microservices deployment.

The per-persona analysis reveals a nuanced interplay between the memorization and generalization branches: the Wide branch contributes most to users with structured, repetitive purchase patterns (Homemaker HR@10 = 0.5260, Drinker = 0.5120), where co-purchase lift rules encode high-confidence associations. Conversely, the Deep branch sustains competitive accuracy for casual shoppers (Casual HR@10 = 0.4320) through SBERT’s semantic cold-start generalization, demonstrating that the architectural decoupling enables each branch to specialize without interfering with the other. The observed coverage-vs-precision trade-off (HR@10 increases while NDCG@10 decreases relative to Deep-Only) reflects the additive scoring mechanism broadening the recommendation set—a desirable property for retail discovery scenarios where surfacing diverse relevant products outweighs strict ranking precision.

B. Future Work

Several concrete research directions emerge to transform the static offline ranker into a dynamic, self-learning recommen-

dation engine:

Contextual Multi-Armed Bandit Re-ranking. The current additive scoring mechanism applies a static weight combination across all inference requests. To capture real-time contextual shifts (e.g., time-of-day dynamics, active promotional events, or flash sales), we propose integrating a LinUCB or Thompson Sampling contextual multi-armed bandit layer above the two-tower outputs. By using the $64d$ normalized user embedding $\mathbf{u}(x)$ concatenated with real-time context signals as the arm feature space, the bandit can dynamically adjust branch weights per request without requiring full model re-training. This mechanism is expected to improve performance for highly volatile segments such as Cluster 3 (Casual Retail, current HR@10 = 0.4320).

Session-Aware Sequence Modeling. While the current User Tower relies on static persona cluster embeddings ($8d$) and historical User IDs ($64d$), it cannot capture short-term intent within an active browsing or chat session. We propose integrating a lightweight SASRec-lite module (a 1–2 layer Transformer with causal self-attention and hidden dimension ≤ 64) directly into the User Tower. The module will process the sequence of the last 5 interacted SKUs in real time, projecting short-term intent into the same $64d$ embedding space while strictly bounding inference overhead to maintain the sub-millisecond ONNX serving guarantee (~ 0.85 ms).

End-to-End Joint Fine-Tuning. The current Item Tower utilizes frozen SBERT embeddings to guarantee zero-shot cold-start coverage. A promising extension involves gradually unfreezing the top-2 Transformer layers of the Vietnamese SBERT model during joint training. Using discriminative learning rates (10^{-5} for language model layers vs. 10^{-3} for MLP heads), the model can learn domain-specific co-purchase semantics (e.g., aligning “grilled squid” with “canned beer” in vector space) to improve ranking precision (NDCG@10), while relying on fallback representations for zero-history items.

REFERENCES

- [1] S. Zhang, L. Yao, A. Sun, and Y. Tay, “Deep learning based recommender system: A survey and new perspectives,” *ACM Computing Surveys*, vol. 52, no. 1, 2019.
- [2] P. Covington, J. Adams, and E. Sargin, “Deep neural networks for YouTube recommendations,” in *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys)*, 2016.
- [3] R. Agrawal and R. Srikant, “Fast algorithms for mining association rules in large databases,” in *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB)*, 1994, pp. 487–499.
- [4] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, “Item-based collaborative filtering recommendation algorithms,” in *Proceedings of the 10th International Conference on World Wide Web (WWW)*, 2001.
- [5] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, “Neural collaborative filtering,” in *Proceedings of the 26th International Conference on World Wide Web (WWW)*, 2017.
- [6] H.-T. Cheng, L. Koc, J. Harmsen, T. Shaked, T. Chandra, H. Aradhye, G. Anderson, G. Corrado, W. Chai, M. Ispir, R. Anil, Z. Haque, L. Hong, V. Jain, X. Liu, and H. Shah, “Wide & deep learning for recommender systems,” in *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems (DLRS)*, 2016.
- [7] H. Guo, R. Tang, Y. Ye, Z. Li, and X. He, “DeepFM: A factorization-machine based neural network for CTR prediction,” in *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*, 2017.
- [8] R. Wang, B. Fu, G. Fu, and M. Wang, “Deep & cross network for ad click predictions,” in *Proceedings of the ADKDD Workshop at KDD*, 2017.
- [9] S. Rendle, “Factorization machines,” in *Proceedings of the 10th IEEE International Conference on Data Mining (ICDM)*, 2010.
- [10] X. Yi, J. Yang, L. Hong, D. Z. Cheng, L. Heldt, A. Kumthekar, Z. Zhao, L. Wei, and E. Chi, “Sampling-bias-corrected neural modeling for large corpus item recommendations,” in *Proceedings of the 13th ACM Conference on Recommender Systems (RecSys)*, 2019.
- [11] J. Yang, X. Yi, D. Z. Cheng, L. Hong, Y. Li, S. X. Wang, T. Xu, and E. H. Chi, “Mixed negative sampling for learning two-tower neural networks in recommendations,” in *Proceedings of The Web Conference (WWW)*, 2020.
- [12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019.
- [13] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.
- [14] K. Jarvelin and J. Kekalainen, “Cumulated gain-based evaluation of IR techniques,” *ACM Transactions on Information Systems (TOIS)*, vol. 20, no. 4, pp. 422–446, 2002.
- [15] W. Krichene and S. Rendle, “On sampled metrics for item recommendation,” *Communications of the ACM*, vol. 65, no. 7, pp. 75–83, 2022.
- [16] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.
- [17] M. Volkovs, G. Yu, and T. Poutanen, “DropoutNet: Addressing cold start in recommender systems,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.